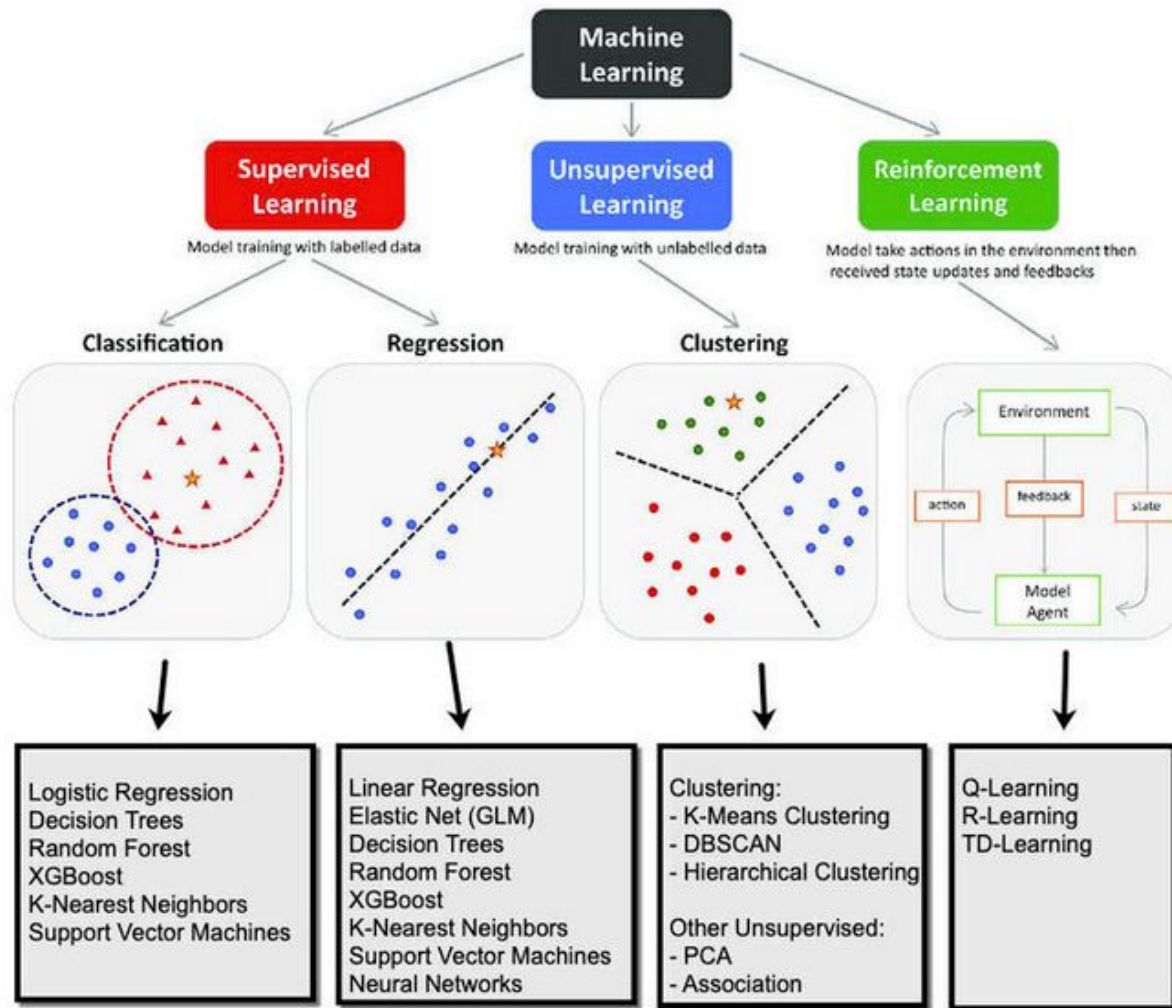


Classical Machine Learning

Supervised Learning

Week 4 - Day 1



What is Supervised Learning?

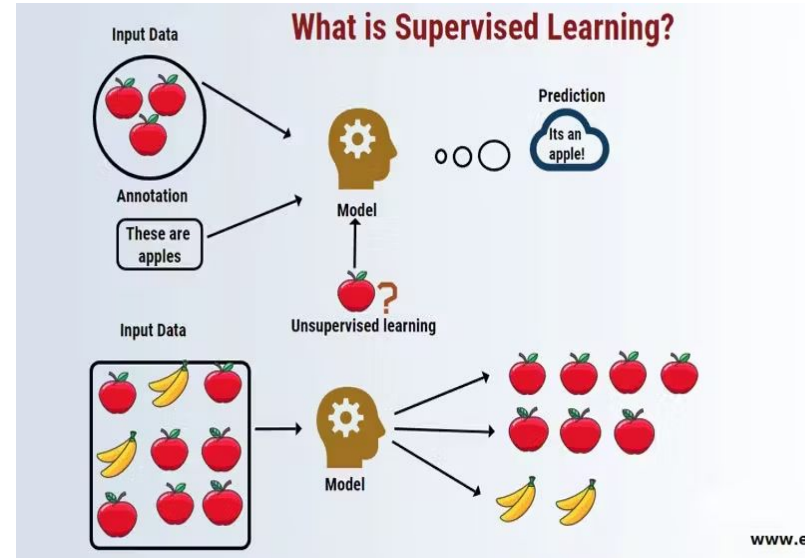
- Supervised learning is a type of machine learning where the model is trained on labeled data.
- Each training example includes an input and the correct output.

Key Characteristics:

- Input → Model → Output
- Goal: Learn a mapping function from inputs to outputs

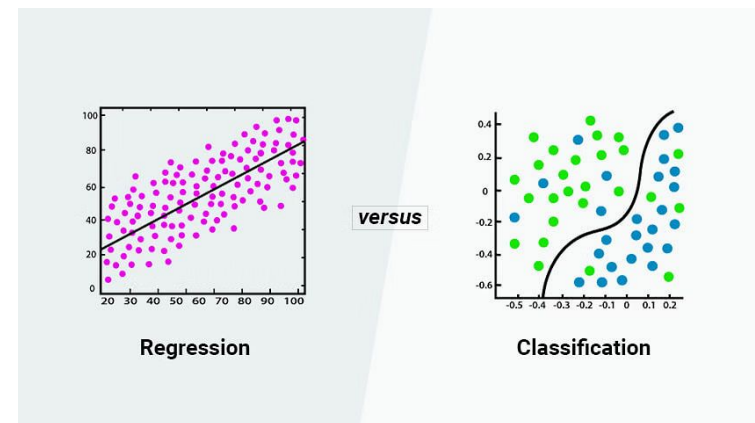
Types:

- **Regression** – Predicts continuous values
- **Classification** – Predicts discrete class labels



Regression vs Classification

Aspect	Regression	Classification
Output Type	Continuous (Real values)	Discrete (Categories/Labels)
Example	Predict house price	Spam detection (Spam/Not Spam)
Metric	MSE, RMSE, R^2 Score	Accuracy, Precision, Recall



Key Insight:

Choose regression when predicting numbers; classification when predicting labels.

Linear Regression

A linear approach to modeling the relationship between a dependent variable and one or more independent variables.

Equation:

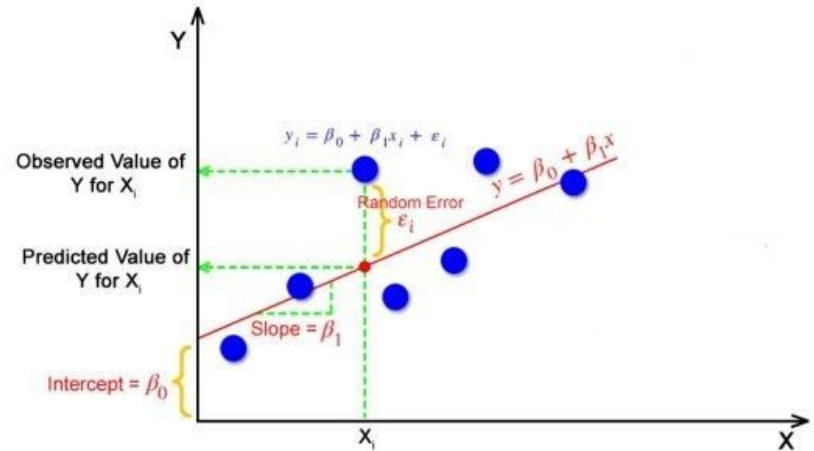
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Goal:

Minimize the residual sum of squares between observed and predicted values.

Assumption:

Linear relationship between features and target.



Ridge and Lasso Regression

Problem in Linear Regression:

Overfitting and multicollinearity

To avoid this, we use regularization

Regularization Techniques:

1 Ridge Regression (L2 penalty):

Adds $\lambda \sum \beta_i^2$ to the loss function. Shrinks coefficients but keeps all features.

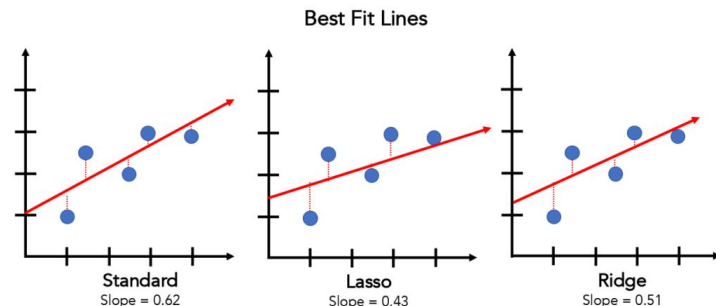
$$\text{Loss} = \text{MSE} + \lambda \sum \beta_i^2$$

- When all features are useful

2 Lasso Regression (L1 penalty):

Adds $\lambda \sum |\beta_i|$ to the loss function. Can reduce some coefficients to zero.

$$\text{Loss} = \text{MSE} + \lambda \sum |\beta_i|$$



Regression Evaluation Metrics

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE):

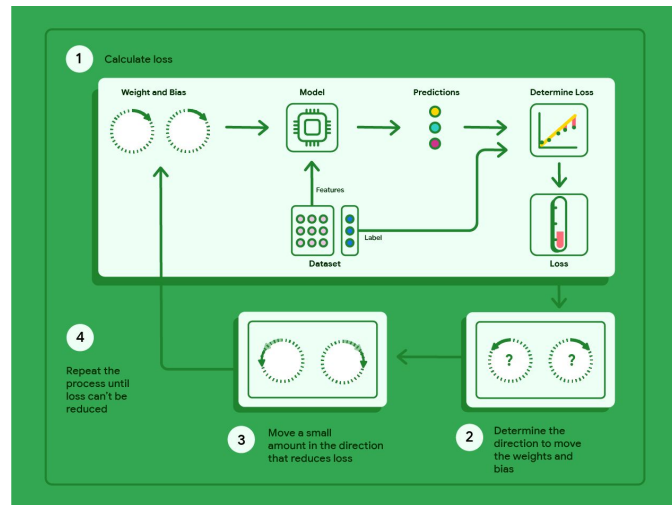
$$\text{RMSE} = \sqrt{\text{MSE}}$$

Gradient descent

Gradient descent is a mathematical technique that iteratively finds the weights and bias that produce the model with the lowest loss. Gradient descent finds the best weight and bias by repeating the following process for a number of user-defined iterations.

The model begins training with randomized weights and biases near zero, and then repeats the following steps:

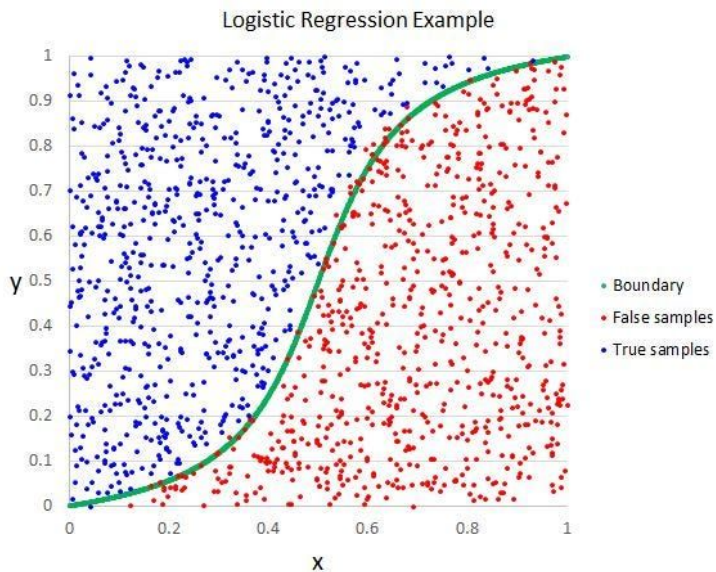
1. Calculate the loss with the current weight and bias.
2. Determine the direction to move the weights and bias that reduce loss.
3. Move the weight and bias values a small amount in the direction that reduces loss.
4. Return to step one and repeat the process until the model can't reduce the loss any further



Logistic Regression

Logistic regression aims to solve classification problems. It does this by predicting categorical outcomes, unlike linear regression that predicts a continuous outcome.

In the simplest case there are two outcomes, which is called **binomial**, an example of which is predicting if a tumor is malignant or benign. Other cases have more than two outcomes to classify, in this case it is called **multinomial**. A common example for multinomial logistic regression would be predicting the class of an iris flower between 3 different species.



Logistic Function

Equation:

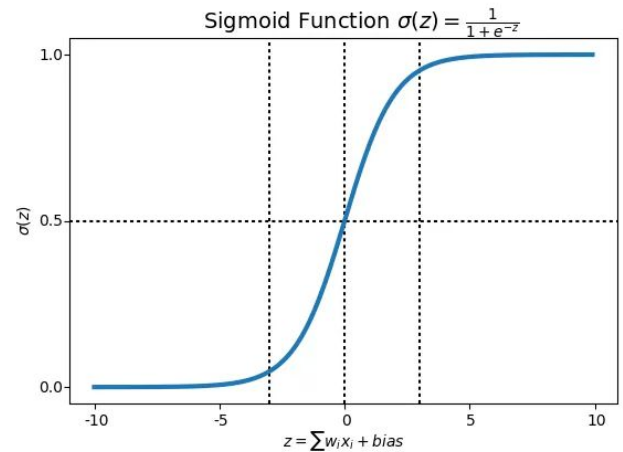
$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)}}$$

Decision Boundary:

- If $P \geq 0.5 \rightarrow$ Class 1
- If $P < 0.5 \rightarrow$ Class 0

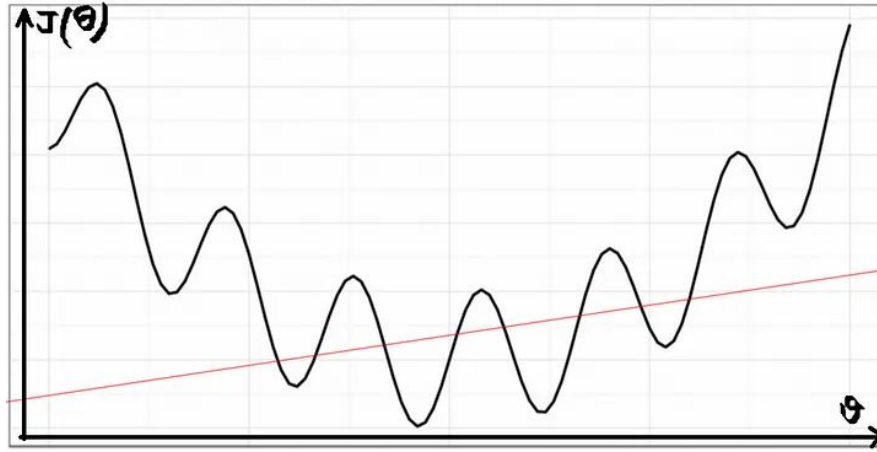
Important Note:

It outputs probabilities, not direct class labels.



Cost Function of Logistic Regression

$$J(\theta) = -\frac{1}{m} \sum \left[y^{(i)} \log(h\theta(x(i))) + (1 - y^{(i)}) \log(1 - h\theta(x(i))) \right]$$

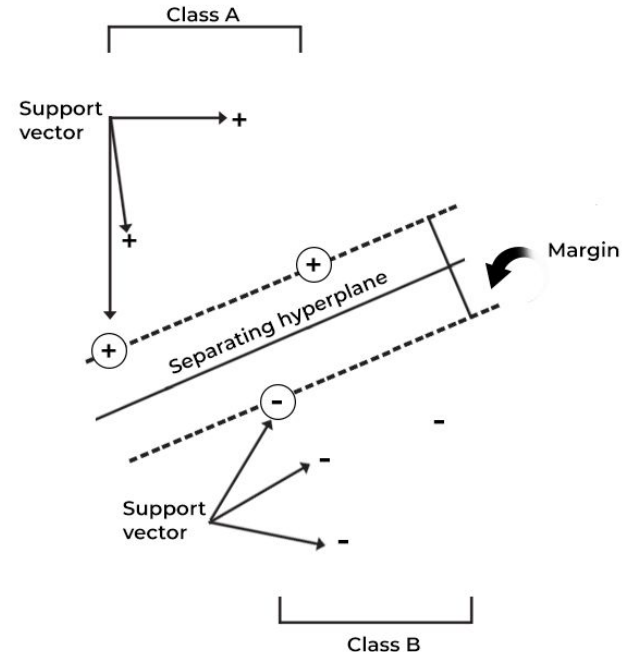


Support Vector Machine (SVM)

A support vector machine (SVM) is a machine learning algorithm that uses supervised learning models to solve complex classification, regression, and outlier detection problems by performing optimal data transformations that determine boundaries between data points based on predefined classes, labels, or outputs

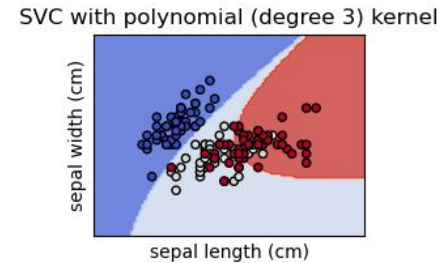
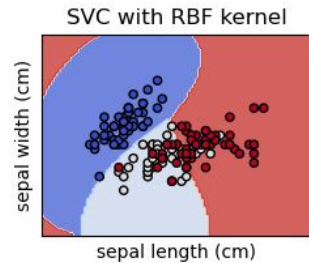
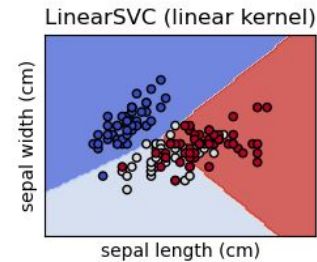
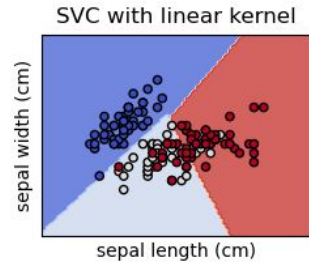


SVMS OPTIMIZE MARGIN BETWEEN SUPPORT VECTORS OR CLASSES



Type of SVMs

1. Simple or linear SVM
2. Kernel or non-linear SVM



Kernel functions of SVM

The *kernel function* can be any of the following:

- linear: $\langle x, x' \rangle$.
- polynomial: $(\gamma \langle x, x' \rangle + r)^d$, where d is specified by parameter `degree`, r by `coef0`.
- rbf: $\exp(-\gamma \|x - x'\|^2)$, where γ is specified by parameter `gamma`, must be greater than 0.
- sigmoid $\tanh(\gamma \langle x, x' \rangle + r)$, where r is specified by `coef0`.

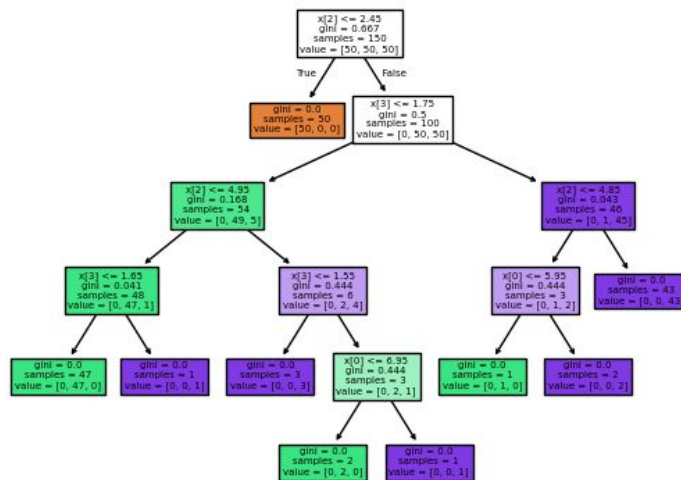
Decision Tree

- A **Decision Tree** is a **flowchart-like** structure used for classification and regression tasks.
- It **splits data** into branches based on conditions/questions.
- Each **internal node** represents a decision based on a feature.
- Each **leaf node** represents a final output or prediction.

Terminology:

- **Root Node** – The first feature used to split the data
- **Decision Node** – A feature-based question
- **Leaf Node** – Final prediction/class
- **Branch** – Outcome of a decision

Decision tree trained on all the iris features



How Does a Decision Tree Learn?

Step 1: Entropy – Measuring Uncertainty

Entropy tells us **how pure or impure** a dataset is.

Formula:

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- S : dataset
- p_i : proportion of class i
- c : number of classes

Interpretation:

- **Entropy = 0** → all samples in one class (pure)
- **Entropy = 1** → evenly split classes (maximum disorder)

Step 2: Information Gain – Choosing the Best Split

Information Gain tells us **how much entropy is reduced** after a split.

Formula:

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \sum_k \frac{|S_k|}{|S|} \cdot \text{Entropy}(S_k)$$

- S : the parent dataset
- S_k : each child subset after the split
- $|S_k|/|S|$: weight of the child based on its size

Mini Example:

Dataset: 10 samples

- 6 "Yes" (class 1), 4 "No" (class 0)

$$\text{Entropy}(\text{parent}) = - \left(\frac{6}{10} \log_2 \frac{6}{10} + \frac{4}{10} \log_2 \frac{4}{10} \right) \approx 0.971$$

Split by Feature X:

- Left child: 4 "Yes", 0 "No" → Entropy = 0
- Right child: 2 "Yes", 4 "No" →

$$= - \left(\frac{2}{6} \log_2 \frac{2}{6} + \frac{4}{6} \log_2 \frac{4}{6} \right) \approx 0.918$$

Information Gain:

$$= 0.971 - \left(\frac{4}{10} \cdot 0 + \frac{6}{10} \cdot 0.918 \right) = 0.971 - 0.5508 = 0.4202$$

 Higher Information Gain → Better Split!